Könyvtári kölcsönzési szolgáltatás

Adatbázis	Mit tárol	Miért itt?
PostgreSQL	members, stock, loans, decisions	transaction, constraint, row lock
MongoDB	catalogue record (szabálytalan mezők)	rugalmas, validated document
Redis	decision cache (TTL 5 perc)	gyors, magától lejár
Neo4j	author, subject, ki mit borrowed	relationship traversal
pgvector + Qdrant	description embedding-ek (420 db)	similarity search, összehasonlítás

1. Mi ez az alkalmazás?

Egy könyvtári kölcsönzési szolgáltatást készítettem, a Library Loan Service-t. Ha egy member kölcsönözni szeretne egy könyvet, a program három decision közül választ: APPROVED, WAITLISTED, ha nincs szabad példány, vagy REJECTED, ha valamelyik szabály sérül. Például a member suspended, unpaid fines van rajta, már három active loan-ja van, vagy túl fiatal a könyvhöz. Ha a válasz nem APPROVED, a program alternatív könyveket ajánl. A feladat az volt, hogy mind az öt adatbázistípust használjam, és mindegyik azt tárolja, amihez a legjobban illik.

2. Az öt adatbázis szerepe

A PostgreSQL a members, a stock, a loans és a decisions adatait tárolja, mert itt kell a transaction és a constraint. Ha ketten egyszerre kérik az utolsó copy-t, a row lock (SELECT … FOR UPDATE) miatt az egyik APPROVED lesz, a másik WAITLISTED. A MongoDB a könyvek catalogue record-jait tartalmazza, mert a valódi rekordok szabálytalanok: a fiction könyveknél a subject_people, a nonfiction könyveknél a subject_places mező kötelező. A $jsonSchema validation a hiányos insertet hibával visszautasítja. A Redis cache: a decision-t öt percig, TTL-lel tárolja. Ha egy member-t suspended-re állítunk, de a key-t nem töröljük, a cache még mindig APPROVED-ot mond, vagyis stale. Ezért a módosítással egy lépésben törlöm a key-t. A Neo4j a graph: author, subject, és hogy ki mit borrowed még. A pgvector és a Qdrant a könyvleírások embedding-jeit tárolja a hasonló könyvek kereséséhez. Szándékosan kétszer építettem meg, hogy összehasonlíthassam őket. A PostgreSQL és a MongoDB a system of record, a többi csak derived copy.

3. Az adatok

A 420 könyv valódi: az Open Library nyílt katalógusából töltöttem le, title, author, description és ISBN adatokkal. A members, a loans, a copy count és a min_age szintetikus, mert a valódi borrower adatok személyes adatok, és nem nyilvánosak. Az age-et a teljes birth_date-ből számolom: aki holnap lesz tizennyolc, az ma még tizenhét. A 19 test case-t a kód előtt írtam meg, és mind passed.

4. Vektorkeresés: pgvector és Qdrant

Mindkét rendszerbe ugyanazok a normalized, 384 dimenziós embedding-ek kerültek (all-MiniLM-L6-v2), cosine distance-szel, ugyanazzal a húsz query-vel. Az HNSW approximate keresés mindkettőnél pontosan megegyezett az exact kereséssel: az ANN recall@10 értéke 1,0, a két rendszer top-10 listája pedig mind a tíz id-ban egyezett. Ez 420 vektornál várható, mert az index gyakorlatilag exact. A labelled quality alacsonyabb, recall@5 0,58 és MRR 0,775, mert a genre label-ek az Open Library crowd-sourced tag-jeiből jönnek, és zajosak. Az a query, amely a vektorkeresést a stock-kal összekapcsolja, pgvectorban egyetlen SQL statement, Qdrantnál két lépés. Az eredmény ugyanaz.

5. A hibaeset

A legérdekesebb hiba a filtered search. Ha a keresést olyan genre-re szűröm, amely a sorok körülbelül öt százalékát adja, a pgvector alapbeállítással átlagosan csak 1,65 sort ad vissza tíz helyett. Az index csak negyven ef_search candidate-et ad, és a filter ezután fut. Ha bekapcsolom az iterative scan-t, mind a tíz sor megjön. A Qdrant payload index-szel alapból jól működik.

6. Összefoglalás

Egy forrásrekord módosítása után a Redis, a vektor és a graph addig stale, amíg refresh-t nem futtatunk; ezt a program észleli (find_stale) és javítja. Két saját hibámat is javítottam és dokumentáltam: egy cached prepared statement plan-t, ami meghamisította a mérést, és egy rossz első adatletöltést. Minden szám a kód kimenetéből származik.